

CAREER ADVANCEMENT PLATFORM

AI-Powered Career Development & Interview Preparation System

Developer	Harishwar S
Project Type	Full-Stack Web Application
Technology Stack	React + TypeScript + Firebase + Google Gemini AI
Date Generated	June 21, 2026

TABLE OF CONTENTS

1. Project Overview & Vision
2. Technology Stack & Dependencies
3. Project Architecture
4. File Structure & Organization
5. Core Features & Functionality
6. AI Agents System
7. Key Source Code Files
8. Database Schema
9. Authentication & Security
10. Setup & Deployment Instructions
11. API Integration
12. User Workflow

1. PROJECT OVERVIEW & VISION

Project Name: Career Advancement Platform
Tagline: "Precision Career Evolution" - Deploy a fleet of 7 specialized AI agents to analyze, prepare, and launch your career with surgical precision.

Project Vision:
This application is a next-generation multi-agent AI system designed to revolutionize career development and professional growth. It combines resume analysis, skill gap identification, personalized learning recommendations, and AI-powered interview preparation into a unified platform.

- Key Objectives:**
- Provide deep resume analysis using AI to identify technical and soft skills
 - Identify skill gaps compared to target job roles
 - Recommend certified courses for skill development
 - Generate realistic interview questions for specific job roles
 - Conduct AI-powered mock interviews with performance evaluation
 - Track user progress and provide personalized career recommendations
 - Offer admin dashboard for platform analytics and user management

2. TECHNOLOGY STACK & DEPENDENCIES

Category	Technology	Purpose
Frontend Framework	React 19.0	UI component library
Language	TypeScript 5.8	Type-safe development
Build Tool	Vite 6.2	Fast bundling and development server
Routing	React Router 7.14	Client-side navigation
Styling	Tailwind CSS 4.1	Utility-first CSS framework
Animations	Framer Motion 12.38	Advanced component animations
Backend/Database	Firebase 12.12	Real-time database and authentication
AI API	Google Gemini API	LLM for content generation and analysis
PDF Processing	PDF-Parse 2.4	Extract text from PDF resumes
PDF Generation	jsPDF 4.2	Generate PDF reports
PDF Rendering	html-to-image 1.11	Convert HTML to image for PDF
Charts	Recharts 3.8	Data visualization
Icons	Lucide React 0.546	Icon library
HTTP Client	Express 4.21	Backend server setup

3. PROJECT ARCHITECTURE

Architecture Overview:

The application follows a modern client-server architecture with three main layers:

Frontend Layer (React + TypeScript):

- Single Page Application (SPA) built with React 19
- Component-based UI with Tailwind CSS styling
- State management through React Context API
- Real-time updates through Firebase listeners
- PDF processing and generation capabilities

Backend Layer (Firebase):

- Firebase Authentication for user management
- Cloud Firestore for real-time database
- Security rules for data access control
- Automatic data synchronization

AI/ML Layer (Google Gemini):

- Multi-agent orchestration system
- 7 specialized AI agents for different tasks
- Schema-based JSON response generation
- Retry logic with quota management
- Model fallback mechanisms

Data Flow:

User Input → React Component → AI Service → Gemini API → Firebase Storage → User Dashboard

4. FILE STRUCTURE & ORGANIZATION

Root Level Files:

File/Folder	Description
package.json	Project dependencies and npm scripts
tsconfig.json	TypeScript compiler configuration
vite.config.ts	Vite build tool configuration
index.html	Main HTML entry point
firebase-applet-config.json	Firebase project configuration
firestore.rules	Firebase security rules for Firestore
DRAFT_firestore.rules	Draft security rules backup
metadata.json	Project metadata
README.md	Project documentation
.gitignore	Git ignore file

src/ Directory Structure:

src/App.tsx: Main application component with routing and authentication context

src/main.tsx: React application entry point

src/index.css: Global CSS styles

src/lib/: Utility functions and configurations

- firebase.ts - Firebase initialization and configuration
- utils.ts - Helper functions and error handlers

src/services/: Business logic and API integrations

- aiService.ts - AI agent orchestration and Gemini API integration

src/components/: Reusable React components

- Header.tsx - Navigation header with user profile

src/pages/: Page components for different routes

- Home.tsx - Landing page with feature showcase
- Login.tsx - User authentication page
- Dashboard.tsx - User main dashboard with statistics
- ResumeAnalysis.tsx - Resume upload and analysis page
- Recommendations.tsx - Skill gap and course recommendations
- Interview.tsx - Mock interview arena
- AdminLogin.tsx - Admin authentication
- AdminDashboard.tsx - Admin analytics and monitoring

public/: Static assets

- _redirects - Netlify redirect configuration

5. CORE FEATURES & FUNCTIONALITY

1. Resume Analysis (Resume Lab)

The application allows users to upload PDF or text resumes for deep AI analysis:

- Extract technical skills, soft skills, education, and experience
- Calculate ATS compatibility score (0-100)
- Identify key strengths and weaknesses
- Provide professional summary
- Download analysis as PDF report

2. Targeted Role Analysis

Analyze resume compatibility for specific job roles:

- Calculate compatibility score for target role
- Identify relevant skills already present
- Highlight missing critical skills
- Provide prioritized course recommendations

3. Skill Gap Identification

Compare user skills against industry standards:

- Identify missing skills for target positions
- Prioritize gaps (High/Medium/Low)
- Explain why each skill is important
- Support for 10+ job roles

4. Personalized Course Recommendations

AI-powered learning path generation:

- Recommend 5-10 certified courses per gap
- Direct links to Coursera, Udemy, LinkedIn Learning
- Course duration, difficulty level, and cost information
- Filter by platform and difficulty
- Cost analysis and ROI information

5. Mock Interview Arena

Realistic interview preparation with AI evaluation:

- Select job role and difficulty level (Beginner/Intermediate/Advanced)
- Generate 8 high-quality interview questions
- 5 Technical + 3 Behavioral questions per session
- 2-minute timer per question
- Real-time video/audio recording capability
- Tab-switch detection for proctoring
- AI-powered answer evaluation and scoring
- Comprehensive interview report with feedback
- Download report as PDF

6. User Dashboard

Central hub for career progress tracking:

- Resume score tracking
- Interview performance statistics
- 7-day performance chart
- Recent resume analyses
- Interview session history
- Quick links to main features

7. Admin Dashboard

Platform analytics and user management:

- Total user statistics
- Resume analysis count
- Interview session analytics
- Performance distribution charts
- User search and filtering
- System health monitoring

6. AI AGENTS SYSTEM

The platform uses 7 specialized AI agents orchestrated through Google Gemini API. Each agent is optimized for specific tasks with tailored prompts and response schemas:

Agent	Purpose	Input	Output
Analyzer Agent	Resume parsing and scoring	Resume text	Skills, experience, score
Gap Agent	Identify missing skills	User skills + target roles	Missing skills list with priority
Course Agent	Recommend learning paths	Missing skills	Course list with links and details
Interview Agent	Generate interview questions	Job role + difficulty	8 interview questions (5 tech + 3 behavioral)
Evaluator Agent	Evaluate interview answers	Question + user answer	Score and feedback
Targeted Role Agent	Role-specific analysis	Resume + target role	Compatibility score and recommendations

Agent Implementation Details:

generateWithRetry Function:

All agents use a robust retry mechanism with:

- Maximum 3 retry attempts
- Exponential backoff (3, 6, 9 seconds)
- Quota management (429 error handling)
- Model fallback (gemini-3-flash-preview → gemini-flash-latest)
- Structured JSON responses using schema validation

API Key Management:

- Loads from VITE_GEMINI_API_KEY environment variable
- Supports both .env and .env.local files
- Graceful error handling for missing keys

7. KEY SOURCE CODE FILES

A. App.tsx (Main Application Component)

- Creates AuthContext for managing user authentication state
- Implements useAuth hook for accessing auth state
- Sets up React Router with 8 main routes
- Monitors Firebase authentication state changes
- Implements protected routes (requires login)
- Tracks admin status separately
- Includes loading state during app initialization

B. lib/firebase.ts (Firebase Configuration)

- Initializes Firebase app with configuration from firebase-applet-config.json
- Exports auth instance for authentication operations
- Initializes Firestore database with experimental features
- Implements connection testing function
- Handles offline and offline-first scenarios

C. services/aiService.ts (AI Orchestration Layer)

Contains 6 main exported functions:

1. **analyzerAgent(resumeText)** - Extracts resume data
Returns: {score, technicalSkills[], softSkills[], experience, education[], summary}
2. **gapAgent(userSkills[], targetRoles[])** - Identifies skill gaps
Returns: [{skill, priority, reason}]
3. **courseAgent(missingSkills[])** - Recommends courses
Returns: [{name, platform, certification, duration, difficulty, cost, link, skillsCovered[]}]
4. **interviewAgent(role, difficulty)** - Generates interview questions
Returns: [{question, category}] where category is 'Technical' or 'Behavioral'
5. **evaluateAnswer(question, answer)** - Evaluates interview answers
Returns: {score (0-10), feedback}
6. **targetedRoleAgent(resumeText, targetRole)** - Analyzes for specific role
Returns: {compatibilityScore, relevantSkills[], missingSkills[], summary}

8. DATABASE SCHEMA (FIRESTORE)

Collections Structure:

1. users Collection

Document ID: user.uid

Field	Type	Description
uid	string	Firebase user ID (unique identifier)
name	string	User full name
email	string	User email address
createdAt	timestamp	Account creation date

2. resumes Collection

Document ID: Auto-generated

Field	Type	Description
userId	string	Reference to user who uploaded resume
score	number	Overall resume score (0-100)
technicalSkills[]	array	List of technical skills identified
softSkills[]	array	List of soft skills identified
experience	string	Years of experience summary
education[]	array	Educational qualifications
summary	string	AI-generated resume summary
content	string	Full resume text (truncated to 10KB)
createdAt	timestamp	Analysis date

3. interviews Collection

Document ID: Auto-generated

Field	Type	Description
userId	string	Reference to user conducting interview
jobRole	string	Target job role
difficulty	string	Difficulty level (Beginner/Intermediate/Advanced)
overallScore	number	Final interview score (0-100)

technicalScore	number	Technical questions score
communicationScore	number	Communication evaluation score
behaviorScore	number	Behavior monitoring score
tabSwitches	number	Number of detected tab switches
violations[]	array	List of proctoring violations
questions[]	array	Question-answer pairs with evaluations
startedAt	timestamp	Interview start time

9. AUTHENTICATION & SECURITY

Authentication Methods:

1. Email/Password Authentication

- User registration with email and password
- Password stored securely by Firebase
- Account creation automatically stores user data in Firestore
- Display name setup during registration

2. Google OAuth Authentication

- Single sign-on via Google account
- Uses Google Sign-In popup
- Automatically creates user profile in Firestore
- Merge user data if account already exists

Security Measures:

Firebase Security Rules (Firestore):

- Users can only read/write their own documents
- Public read access for certain collections
- Authenticated user requirement for sensitive data
- Role-based access for admin operations

Client-Side Security:

- Protected routes requiring authentication
- Admin routes with separate authentication flow
- useAuth hook for permission checking
- Automatic logout on tab close

API Security:

- Environment variables for API keys (not hardcoded)
- Gemini API key stored in .env.local
- Firebase config stored in firebase-applet-config.json
- No sensitive data exposed in client code

Data Privacy:

- Resume content stored only in user's account
- Interview results associated with user ID
- Proctoring data (tab switches) logged for integrity
- User can request data deletion

10. SETUP & DEPLOYMENT INSTRUCTIONS

Prerequisites:

- Node.js 16+ installed
- npm or yarn package manager
- Firebase project created (Google account required)
- Gemini API key from Google AI Studio
- Git for version control

Local Development Setup:

Step 1: Install Dependencies

Command: `npm install`

This installs all packages listed in `package.json` including React, Firebase, Tailwind CSS, and AI libraries.

Step 2: Configure Environment Variables

Create `.env.local` file in project root:

`VITE_GEMINI_API_KEY=your_gemini_api_key_here`

Step 3: Configure Firebase

Update `firebase-applet-config.json` with your Firebase project credentials:

- `apiKey`
- `authDomain`
- `projectId`
- `storageBucket`
- `messagingSenderId`
- `appId`
- `firestoreDatabaseId`

Step 4: Start Development Server

Command: `npm run dev`

Starts Vite dev server on `http://localhost:3000`

Hot Module Replacement (HMR) enabled for instant updates

Production Build:

Command: `npm run build`

Creates optimized production build in `dist/` folder

Output is minified and tree-shaken for minimal size

Deployment Platforms:

Option 1: Netlify (Recommended)

- Connect GitHub repository
- Set build command: `npm run build`
- Set publish directory: `dist`
- Add environment variables in Netlify dashboard
- Deploy automatically on push to main branch

Option 2: Vercel

- Import project from GitHub
- Add environment variables in Vercel dashboard
- Vercel automatically detects Vite config
- Deploy with single click

Option 3: Firebase Hosting

Command: `firebase deploy`

Requires Firebase CLI and project setup

Environment Variables (Production):

`VITE_GEMINI_API_KEY` - Google Gemini API key

`VITE_FIREBASE_CONFIG` - Firebase configuration (can be in JSON)

11. API INTEGRATION

Google Gemini API Integration:

API Endpoint:

<https://generativelanguage.googleapis.com/v1/models/>

Supported Models:

- gemini-3-flash-preview (Primary - faster, more reliable)
- gemini-flash-latest (Fallback)
- gemini-3-pro (Alternative for complex tasks)

API Rate Limits:

- Free tier: 15 requests per minute
- Quota errors (429) trigger exponential backoff
- Automatic retry with 3-second escalation
- Model fallback after 404 errors

Request Format:

```
{ "model": "gemini-3-flash-preview", "contents": "prompt text here", "config": { "responseMimeType":  
"application/json", "responseSchema": { "type": "OBJECT", "properties": {...} } } }
```

Firebase API Endpoints (via SDK):

Authentication:

- signInWithEmailAndPassword(auth, email, password)
- createUserWithEmailAndPassword(auth, email, password)
- signInWithPopup(auth, provider)
- signInOut(auth)
- onAuthStateChanged(auth, callback)

Firestore Database:

- addDoc(collection, data) - Create new document
- getDocs(query) - Retrieve documents
- updateDoc(doc, updates) - Modify document
- deleteDoc(doc) - Delete document
- query(collection, where, orderBy, limit) - Build queries

12. USER WORKFLOW & SYSTEM FLOW

Typical User Journey:

Phase 1: Onboarding

1. User visits home page (Home.tsx)
2. Clicks "Initialize Session" to go to login
3. Either signs up with email/password or uses Google OAuth
4. Automatically redirected to Dashboard (Dashboard.tsx)
5. Profile created in Firestore users collection

Phase 2: Resume Analysis

1. User navigates to "Resume Lab" (ResumeAnalysis.tsx)
2. Uploads PDF or text file containing resume
3. System extracts text from PDF using pdf-parse library
4. Truncates resume to 10KB for AI processing
5. Calls analyzerAgent() from aiService.ts
6. Gemini API returns: score, skills, education, summary
7. Results displayed with interactive UI
8. Analysis stored in Firestore resumes collection
9. User can download report as PDF

Phase 3: Target Role Analysis (Optional)

1. User enters target job role
2. System calls targetedRoleAgent()
3. Returns compatibility score and recommendations
4. Shows relevant skills and missing critical skills
5. Displays course recommendations for gaps

Phase 4: Skill Gaps & Recommendations

1. User navigates to "Smart Recommendations" (Recommendations.tsx)
2. System loads last uploaded resume from Firestore
3. Calls gapAgent() to identify skill gaps
4. Calls courseAgent() to get course recommendations
5. Displays gaps by priority (High/Medium/Low)
6. Shows certified courses with direct links
7. Provides course duration, difficulty, and cost

Phase 5: Mock Interview

1. User navigates to "Mock Arena" (Interview.tsx)
2. Selects target job role and difficulty level
3. Clicks "Start Interview"
4. System calls interviewAgent()
5. Receives 8 questions (5 Technical + 3 Behavioral)
6. Browser requests camera/microphone access
7. User has 2 minutes per question
8. Tab-switch detection monitors exam integrity
9. For each answer, evaluateAnswer() is called
10. AI provides score (0-10) and feedback
11. After all questions, final score calculated
12. Interview result stored in Firestore
13. User can download comprehensive report as PDF

Phase 6: Dashboard Tracking

1. User visits Dashboard (Dashboard.tsx)
2. Displays resume scores, interview stats
3. 7-day performance chart updated with latest data
4. Recent resume analyses and interview sessions listed
5. Quick links to main features available

Admin Workflow:

1. Admin visits /admin/login (AdminLogin.tsx)

2. Authenticates with special admin account
3. Redirected to Admin Dashboard (AdminDashboard.tsx)
4. Views platform statistics:
 - Total users registered
 - Total resumes analyzed
 - Total interview sessions
5. Views performance distribution charts
6. Can search and filter users
7. Can export audit logs

13. TECHNICAL HIGHLIGHTS & ADVANCED FEATURES

1. PDF Processing Pipeline

- Accepts both PDF and text file uploads
- Uses pdf-parse library to extract text from PDFs
- Handles multi-page documents (limits to first 10 pages)
- Truncates to 10KB for optimal AI processing
- Graceful error handling with user-friendly messages

2. Real-Time Database Synchronization

- Firestore listeners update UI in real-time
- Data persists across browser sessions
- Offline-first architecture with local caching
- Experimental long polling for better connectivity

3. Advanced Interview Proctoring

- Tab-switch detection monitors exam integrity
- Records tab switch timestamps and violations
- Behavior score penalty for suspicious activity
- Video/audio capture capability (browser dependent)
- Countdown timer with auto-advance feature

4. Multi-Stage PDF Report Generation

- Converts React component to image using html-to-image
- Generates PDF from image using jsPDF
- Maintains styling and formatting in PDF
- Download with timestamped filename
- High quality (2x pixel ratio) output

5. AI Agent Orchestration

- Modular agent functions for separation of concerns
- Schema-based response validation
- Automatic model fallback on errors
- Exponential backoff retry logic
- Structured error tracking with context

6. Component Animation System

- Framer Motion for smooth transitions
- Staggered animations for lists
- Conditional animations based on state
- Motion contexts for performance optimization
- Page transition effects

7. Responsive Design

- Mobile-first Tailwind CSS approach
- Breakpoints: sm (640px), md (768px), lg (1024px)
- Touch-friendly interface elements
- Adaptive layouts for all screen sizes
- CSS Grid and Flexbox for modern layouts

8. Error Handling Strategy

- Try-catch blocks with graceful fallbacks
- User-friendly error messages
- Error context with operation type and user info
- Firestore error logging for debugging
- API fallback mechanisms

14. PERFORMANCE OPTIMIZATION & BEST PRACTICES

Build Optimization:

- Tree-shaking eliminates unused code
- Code splitting with React lazy loading
- Minification reduces bundle size
- CSS optimization with Tailwind purging
- Asset compression for images and fonts

Runtime Performance:

- useContext for state management (no Redux overhead)
- Memoization with React.memo for components
- useCallback for event handler optimization
- Lazy loading of pages with React.lazy()*
- Image optimization with lazy loading

API Optimization:

- Quota management with backoff strategy
- Response caching in Firestore
- Batch operations for multiple documents
- Limited query results (orderBy + limit)
- Connection pooling via Firebase SDK

Database Optimization:

- Indexed queries for better performance
- Document design to avoid subcollections
- Pagination support with cursor-based navigation
- Data denormalization where appropriate
- Archive old data to reduce collection size

Security Best Practices:

- Never store API keys in client code
- Use environment variables for secrets
- HTTPS for all communications
- Firebase security rules enforce access control
- Regular security updates of dependencies

15. FUTURE ENHANCEMENTS & ROADMAP

Planned Features:

Phase 2 Enhancements:

1. Real-time video interview with AI proctoring
 - Facial recognition for identity verification
 - Eye-gaze detection to monitor attention
 - Background verification
2. Advanced analytics dashboard
 - Performance trends over time
 - Comparative analysis with industry standards
 - Skill growth tracking
 - Job market insights
3. Social networking features
 - User profiles and portfolios
 - Skill endorsements from peers
 - Networking recommendations
 - Discussion forums
4. Integration with job platforms
 - LinkedIn API integration
 - Job board integration (Indeed, Glassdoor)
 - Resume submission automation
 - Application tracking system
5. Advanced AI capabilities
 - Voice-to-text transcription
 - Real-time language translation
 - Sentiment analysis of answers
 - Personalized learning paths using machine learning
6. Enterprise features
 - Team management and collaboration
 - Custom assessment creation
 - White-label solution
 - API for third-party integrations
 - Advanced reporting and analytics

Technical Debt & Improvements:

- Migrate to TypeScript strict mode
- Add comprehensive unit and integration tests
- Implement error boundary components
- Add service worker for offline support
- Implement advanced state management (Redux Toolkit or Zustand)
- Add performance monitoring (Sentry or similar)
- Increase code coverage to 80%+
- Implement E2E testing with Cypress or Playwright

16. DEPLOYMENT & MONITORING

Current Deployment Status:

The application is deployed on Netlify with the following configuration:

Netlify Build Settings:

- Build Command: npm run build
- Publish Directory: dist
- Node Version: 18.x or higher
- Build Timeout: 15 minutes

Environment Variables (Netlify):

- VITE_GEMINI_API_KEY - Google Gemini API key
- All Firebase config loaded from firebase-applet-config.json

Monitoring & Analytics:

Performance Monitoring:

- Vite dev server provides instant feedback
- Production build size analysis with vite-plugin-visualizer
- Core Web Vitals tracking recommended
- Bundle size monitoring

Error Tracking:

- Browser console logging for debugging
- Firebase console for database errors
- Cloud logging for API errors
- Recommended: Sentry integration for production

Usage Analytics:

- Firestore provides query insights
- Admin dashboard shows user statistics
- Resume analysis metrics
- Interview session analytics
- Daily active user tracking

Health Checks:

- Firebase connection test on app startup
- API key validation
- Database accessibility verification
- Authentication service status

Backup & Recovery:

- Firestore automatic daily backups (Google managed)
- Source code version control on GitHub
- Environment variables backed up in CI/CD platform
- User data export capability (GDPR compliance)

17. CONCLUSION & SYSTEM SUMMARY

Project Summary:

The Career Advancement Platform represents a cutting-edge application of AI agents to career development. By combining resume analysis, skill gap identification, personalized learning recommendations, and interactive interview preparation, the platform provides a comprehensive career development solution.

Key Achievements:

- ✓ Full-stack modern web application with React 19
- ✓ 7 specialized AI agents for career analysis
- ✓ Real-time database with Firebase Firestore
- ✓ Secure authentication with multiple methods
- ✓ Advanced interview simulation with proctoring
- ✓ PDF processing and report generation
- ✓ Responsive design for all devices
- ✓ Admin dashboard for platform monitoring
- ✓ Production-ready deployment on Netlify

Technical Excellence:

- Type-safe development with TypeScript
- Component-based architecture with React
- Real-time synchronization with Firestore
- Robust error handling and retry mechanisms
- Performance optimized for production
- Security best practices throughout
- Responsive and accessible UI
- Scalable agent-based architecture

Business Impact:

- Helps professionals identify skill gaps
- Provides personalized learning paths
- Reduces interview anxiety with practice
- Increases job search effectiveness
- Measurable career advancement tracking
- Cost-effective alternative to expensive coaching
- Scalable to enterprise customers

Next Steps for Development:

1. Add comprehensive testing suite
2. Implement real-time video interview features
3. Integrate with major job platforms
4. Build team collaboration features
5. Add advanced analytics dashboard
6. Implement white-label solution
7. Create mobile native apps
8. Build enterprise features

Project Maintenance:

- Regular dependency updates
- Monthly security audits
- Quarterly feature releases
- Continuous performance monitoring
- User feedback incorporation
- Documentation maintenance

APPENDIX A: USEFUL COMMANDS & REFERENCES

Development Commands:

npm install - Install all dependencies
npm run dev - Start development server on port 3000
npm run build - Build for production
npm run preview - Preview production build locally
npm run clean - Remove dist folder
npm run lint - Run TypeScript type checking

Git Commands:

git clone [repo-url] - Clone repository
git add . - Stage all changes
git commit -m "message" - Commit changes
git push origin main - Push to GitHub
git pull - Update local from remote

Firebase CLI Commands:

firebase init - Initialize Firebase project
firebase serve - Run emulator locally
firebase deploy - Deploy to Firebase
firebase login - Authenticate with Firebase

Useful Links:

- React Documentation: <https://react.dev>
- TypeScript Handbook: <https://www.typescriptlang.org/docs>
- Firebase Documentation: <https://firebase.google.com/docs>
- Tailwind CSS: <https://tailwindcss.com/docs>
- Vite Guide: <https://vitejs.dev/guide>
- Google Gemini API: <https://ai.google.dev>